

# WEEK 01

## Threads & Concurrency Basics

## Problem Set

*Four problems. Warm up, then make some mayhem.*

*This week is about causing race conditions on purpose, watching them with your own eyes, and writing down what you saw.*

***We do not fix anything this week. That is next week's job.***

### PROBLEMS

- 01** Run, observe, write down
- 02** Make the race worse
- 03** Parallel word count
- 04** Multi-threaded simulation of your choice

WARM UP  
REQUIRED  
RECOMMENDED  
STRETCH

## OVERVIEW

# The week, in four problems.

Four problems for the week. They start gentle and get more open-ended. The goal is to see concurrency bugs happen with your own eyes — observations matter as much as code.

| #  | Problem                   | Type              | Required?   |
|----|---------------------------|-------------------|-------------|
| 01 | Run, observe, write down  | Warm-up           | Yes         |
| 02 | Make the race worse       | Code modification | Yes         |
| 03 | Parallel word count       | From scratch      | Recommended |
| 04 | Multi-threaded simulation | Open-ended        | Stretch     |

**Before you start:** work through `theory.pdf` and run the examples in `code/` at least once. The problems below assume you've seen the example programs behave (and misbehave).

## How to compile and run

All four example programs in `code/` can be built with the provided Makefile:

```
$ cd week-1/code
$ make           # builds all four examples
$ ./race_counter
$ make run-all  # builds and runs everything
$ make clean    # remove compiled binaries
```

Or manually: `gcc -Wall -pthread 01_hello_thread.c -o hello_thread`

## PROBLEM 01

# Run, observe, write down

## WARM-UP · REQUIRED

This problem doesn't have you write much new code. It is the foundation for everything else.

## What to do

For each of the four examples in code/ — `hello_thread`, `race_counter`, `parallel_sum`, `bank_chaos`:

1. Compile it.
2. Run it at least **10 times** in a row. Use a small shell loop if you like:

```
$ for i in {1..10}; do ./race_counter; done
```

3. For `race_counter`, `parallel_sum`, and `bank_chaos`, write down (in your `REPORT.md`):

- The expected output
- The range of actual outputs you saw across runs
- Whether the output was ever correct, and how often

## For `race_counter` specifically

Also try these variations and note what happens:

- Change `ITERS` to 1000 (very small). Does the bug still show up? Why might it not?
- Change `ITERS` to 1000000000 (one hundred million). Does the bug show up more reliably?
- Compile with `-O2` (`gcc -Wall -O2 -pthread ...`). Does the behavior change?

## What to submit

A file `submissions/<your-handle>/problem-1/observations.md` (or fold these observations into your main `REPORT.md`). No new C code required for this problem — it's about looking carefully.

## PROBLEM 02

# Make the race worse

## REQUIRED

Take the race condition in `02_race_counter.c` and **make it more reliable, on purpose**.

A race condition has to "win" a tiny window of time between the load and the store. Sometimes that window is so small that the race rarely manifests. Your job: maximize how often the race happens. Without changing what the code is *trying* to do, modify it so the wrong answer comes up much more reliably.

## Hints (in increasing order of usefulness — don't read further than you need)

## HINT 1

Race conditions are about the *gap* between the load and the store. If you can make that gap longer, the race becomes much more likely.

## HINT 2

Split `counter++` into its three logical pieces manually:

```
int tmp = counter;
// ... insert something here ...
tmp = tmp + 1;
counter = tmp;
```

What could you put in the gap that lets the OS more easily switch to the other thread, or that just gives the other thread time to do its own load?

## HINT 3 (the actual technique)

Try inserting a tiny no-op between the read and the write: a small busy loop, or a `usleep` from `<unistd.h>` (a few microseconds). Be careful with `printf` — it does its own synchronization internally and may suppress the race rather than amplify it.

## What to submit

1. The modified code as `submissions/<your-handle>/problem-2/race_amplified.c`
2. A short explanation in your report of what you changed and why it makes the race more visible.

**Success criterion:** you should be able to get the final counter to be wildly different from the expected value — possibly even close to just `ITERS`, not `2 * ITERS`, if the threads are stomping each other really hard.

## PROBLEM 03

# Parallel word count

## RECOMMENDED

This one is real. Write a program that counts the total number of words in a text file using multiple threads.

## Requirements

Your program takes a filename and a number of threads as command-line arguments:

```
./word_count <filename> <num_threads>
```

- Read the file into memory.
- Divide the file's contents into roughly equal chunks, one per thread.
- Each thread counts the words in its chunk.
- The main thread collects the partial counts and prints the total.

Define a "word" however you like, as long as you document it. A simple definition: a maximal run of non-whitespace characters.

*For this problem, do NOT worry about the race condition on the total. In fact, deliberately make it racy — have each thread add directly to a shared total, the way 03\_parallel\_sum.c does. The point is to see the race appear in real work.*

## How to test

1. Run your program with 1 thread, 2 threads, 4 threads, and 8 threads on a large text file. Try a free book from [Project Gutenberg](https://www.gutenberg.org/) (<https://www.gutenberg.org/>) — *Moby-Dick* is a fine choice and runs to about 215,000 words.
2. Compare to the output of `wc -w <filename>`.
3. Note whether your output matches, how it differs with more threads, and how often.

## Bonus

If you want to also include a *correct* version using a per-thread accumulator (each thread sums into its own local variable, then main adds those up at the end after the threads join), go ahead — that's a great preview of next week.

## What to submit

submissions/<your-handle>/problem-3/word\_count.c plus a note in your report describing the file you tested on and the results across thread counts.

## PROBLEM 04

# Multi-threaded simulation

## STRETCH

Build a small simulation in which multiple threads interact with some shared state in a way that *should* preserve an invariant, but doesn't. Document the invariant, run the simulation, and show how it breaks.

## Ideas (pick one, or invent your own)

**Bank with transfers.** Simulate accounts and threads doing `transfer(from, to, amount)`. The invariant: the total money across all accounts should never change. Watch it change.

**Ticket booking.** A venue with a fixed number of seats; many threads representing booking agents each try to claim seats. The invariant: total seats sold  $\leq$  seats available. Watch it get violated.

**Inventory system.** A warehouse with items; threads representing customers buy items. Invariant: stock never goes negative. Watch it.

**Anything else of your own.** Invent your own scenario. Build it. Break it.

*The point of this problem is to internalize that race conditions are general: they apply to any concurrent code that operates on shared state in multiple steps, not just to `counter++`. Be creative.*

## What to submit

Your code in `submissions/<your-handle>/problem-4/` plus a note in your report explaining the simulation, the invariant, and how you observed it breaking.

## A NOTE ON YOUR MACHINE

### Concurrency bugs depend on hardware

Race conditions can be sensitive to your hardware and OS. If you're running these on a beefy multi-core Linux box, you'll see races almost immediately. On a single-core VM, on macOS, or under some virtualization setups, the same code might appear to behave more "correctly" because the OS happens to schedule things in a less-bad order most of the time.

If you're not seeing races even with very large iteration counts, mention that in your report. Don't fake it — the answer might genuinely be "this machine is unusually lucky," which is itself an interesting observation worth discussing in mentorship sessions.

***When you're done,** see the submission section of `week-1/README.md` for how to package and submit your work — either as a pull request from your fork or as a branch in this repo if you've been added as a collaborator.*